

Data Structure : Assignment 2

1. What is a singly linked list?

A **singly linked list** is a linear data structure where each node contains:

- **Data** (stores the actual value)
- **Pointer (next)** (stores the address of the next node in the sequence)

The last node points to None, indicating the end of the list.

2. How is memory allocated in a linked list?

Memory is **dynamically allocated** when a new node is created. Python's garbage collector handles memory deallocation automatically.

Example:

```
node = Node(5) # Memory allocated dynamically
```

3. Describe the process of inserting a node at the beginning of a linked list.

- Create a new node.
- Set its next pointer to the current head.
- Update the head pointer to the new node.

Example:

```
new_node.next = head  
head = new_node
```

4. What is the time complexity of searching in a linked list?

- **Worst-case:** $O(n)$ (traverse the entire list)
 - **Best-case:** $O(1)$ (if the element is at the head)
-

5. How can you implement a stack using a linked list?

A stack (LIFO) can be implemented using a linked list by:

- **Push operation:** Insert at the beginning.
 - **Pop operation:** Remove from the beginning.
-

6. What are the advantages of a linked list-based stack?

- **Dynamic Size:** No fixed limit like arrays.
 - **Efficient Insertions/Deletions:** Both push and pop run in $O(1)$.
 - **No Memory Wastage:** Uses only the required memory.
-

7. How is a queue implemented using a linked list?

A queue (FIFO) is implemented with:

- A **front pointer** (first element).
- A **rear pointer** (last element).

Operations:

- **Enqueue:** Insert at the rear.
 - **Dequeue:** Remove from the front.
-

8. Explain the process of deleting a node from a linked list.

- Find the node to be deleted.
- Update the previous node's next pointer to skip the node.
- Free the memory of the deleted node.

Example:

```
def delete_node(self, key):  
    temp = self.head  
    if temp is not None:  
        if temp.data == key:  
            self.head = temp.next  
            temp = None  
            return  
        while temp is not None:
```

```
if temp.data == key:
    break

prev = temp
temp = temp.next
if temp is None:
    return

prev.next = temp.next
temp = None
```

9. What is polynomial addition using linked lists?

Polynomial terms are represented as nodes with:

- **Coefficient**
- **Exponent**

Addition involves merging two polynomials by adding terms with the same exponent.

Algorithm for Polynomial Addition Using Linked Lists

1. **Initialize pointers** to the heads of both polynomials.
2. **Traverse both lists** simultaneously:
 - If **p1.exp > p2.exp**, add p1 to result and move p1 forward.
 - If **p1.exp < p2.exp**, add p2 to result and move p2 forward.
 - If **exponents are equal**, add coefficients, insert the sum, and move both pointers.
3. **Insert remaining terms** from p1 or p2.
4. **Return the result polynomial.**

10. How are sparse matrices represented using linked lists?

A **sparse matrix** contains mostly zero values. It is represented using a linked list by storing only non-zero elements along with their row and column indices.

Each node contains:

- **Row index**

- **Column index**
 - **Value**
 - **Pointer to the next node**
-

11. What is a doubly linked list?

A **doubly linked list** is a linked list where each node has:

- **Data**
- **Pointer to the next node** (next)
- **Pointer to the previous node** (prev)

This allows traversal in both directions.

12. How do you insert a node in a doubly linked list?

- Create a new node.
- Adjust the prev and next pointers of the surrounding nodes.

Example (Insert at the beginning):

```
def add_at_head(self, new_data):  
    new_node = Node(new_data)  
    new_node.next = self.head  
    if self.head:  
        self.head.prev = new_node  
    self.head = new_node
```

13. What is dynamic storage management?

Dynamic storage management refers to allocating and deallocating memory at runtime, allowing efficient use of memory.

In Python, this is handled by:

- **Heap memory allocation** for objects.

- **Garbage collection** for unused objects.
-

14. Describe a circular linked list.

A **circular linked list** is a linked list where:

- The last node points to the first node instead of None.
- This allows continuous traversal in a loop.

It can be **singly** or **doubly** linked.

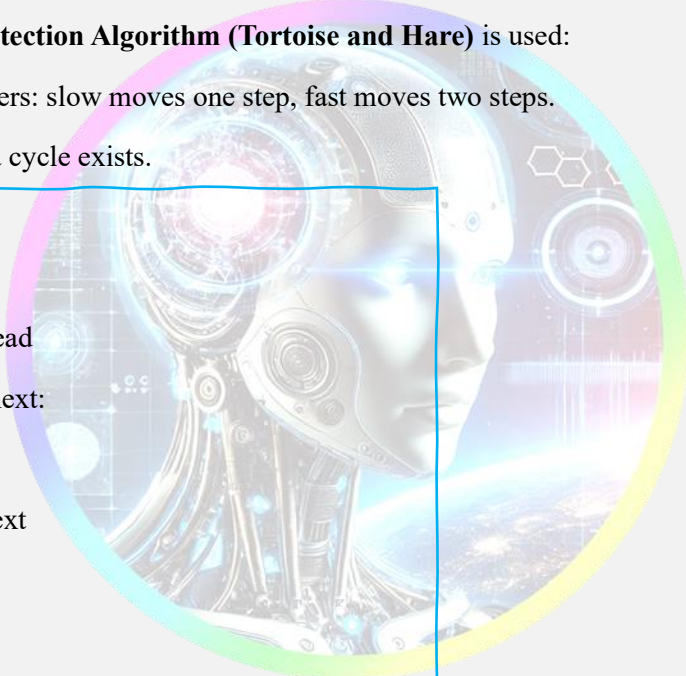
15. How do you detect a cycle in a linked list?

The **Floyd's Cycle Detection Algorithm (Tortoise and Hare)** is used:

- Use two pointers: slow moves one step, fast moves two steps.
- If they meet, a cycle exists.

Example:

```
def has_cycle(head):  
    slow, fast = head, head  
    while fast and fast.next:  
        slow = slow.next  
        fast = fast.next.next  
        if slow == fast:  
            return True  
    return False
```



16. How do you define a node in a singly linked list in Python?

```
class Node:  
    def __init__(self, data):  
        self.data = data  
        self.next = None
```

17. Write Python code to create a singly linked list with three nodes.

```
head = Node(1)
second = Node(2)
third = Node(3)

head.next = second
second.next = third
```

18. How do you insert a node at the beginning of a singly linked list?

- Create a new node.
- Set its next pointer to the current head.
- Update the head pointer.

Example:

```
def add_at_head(self, new_data):
    new_node = Node(new_data)
    new_node.next = self.head
    self.head = new_node
```

19. What is the time complexity of traversing a singly linked list?

- **Worst-case:** $O(n)$ (visit all nodes).
- **Best-case:** $O(1)$ (head contains required data).

20. What is the time complexity of inserting a node at the end of a singly linked list?

- **If the tail pointer is available:** $O(1)$
 - **If only head is available:** $O(n)$ (must traverse to the last node)
-

21. What is the time complexity of deleting a node from a singly linked list?

- **Best case** (delete head): $O(1)$
 - **Worst case** (delete last or search first): $O(n)$
-

22. How do you reverse a singly linked list?

- Initialize three pointers: `prev = None`, `current = head`, and `next = None`.
- Traverse the list, reversing the next pointer of each node.
- Update head to the last processed node.

Time Complexity: $O(n)$

Example:

```
def reverse(self):  
    prev = None  
    curr = self.head  
    while curr:  
        next_node = curr.next # Store next node  
        curr.next = prev # Reverse the link  
        prev = curr # Move prev forward  
        curr = next_node # Move curr forward  
    self.head = prev # Update head to the new first node
```



23. What is the advantage of using a linked list over an array?

- **Dynamic size** (no need to predefine size).
 - **Efficient insertions/deletions** (no shifting required).
 - **Better memory utilization** (no wasted space like arrays).
-

24. What is the disadvantage of using a linked list over an array?

- **More memory usage** (extra pointer per node).
- **Slower access time** (no direct indexing, $O(n)$ search).

- **Cache inefficiency** (nodes are scattered in memory).
-

25. How do you implement a stack using a linked list in Python?

- Use a linked list where the **top** of the stack is the head node.
 - **Push:** Insert at the beginning.
 - **Pop:** Remove from the beginning.
-

26. What is the time complexity of pushing an element onto a linked stack?

- **$O(1)$** (insertion at the head)
-

27. What is the time complexity of popping an element from a linked stack?

- **$O(1)$** (removal from the head)
-

28. How do you implement a queue using a linked list in Python?

- Maintain two pointers: front (head) and rear (tail).
 - **Enqueue:** Insert at the rear.
 - **Dequeue:** Remove from the front.
-

29. What is the time complexity of enqueueing an element in a linked queue?

- ☐ Since we maintain a rear pointer, **insertion happens in constant time.**
 - ☐ No shifting of elements is required like in an array.
 - **$O(1)$** (insertion at the rear)
-

30. What is the time complexity of dequeuing an element from a linked queue?

- ☐ Since we maintain a front pointer, we simply move it to the next node.
 - ☐ No need to shift elements, making it an efficient operation.
 - **$O(1)$** (removal from the front)
-

31. How do you represent a polynomial using a linked list?

Each term of the polynomial is stored in a node with:

- **Coefficient** (value of the term)
- **Exponent** (power of the variable)
- **Pointer to the next node**

Example

```
def print_poly(self):  
    temp = self.head  
    while temp:  
        print(f'{temp.coeff}x^{temp.exp}', end=" + ")  
        temp = temp.next  
    print("None")
```

output:

Polynomial:

$3x^2 + 5x^1 + 2x^0 + \text{None}$

32. What is the time complexity of adding two polynomials using linked lists?

- Both polynomials are traversed once.
 - Merging two sorted lists takes **$O(n+m)$** , where n and m are the number of terms in each polynomial.
- ✅ Time Complexity: **$O(n+m)$**

33. What is the difference between a singly linked list and a doubly linked list?

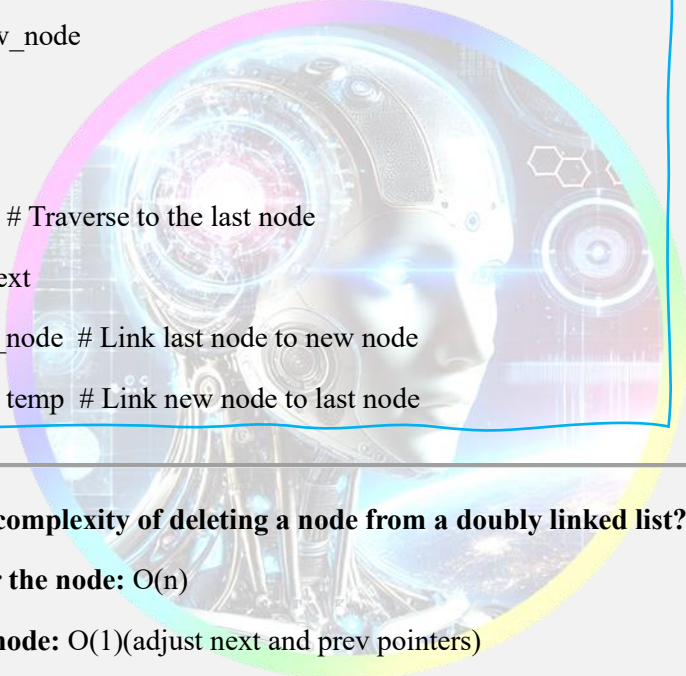
Feature	Singly Linked List	Doubly Linked List
Pointers	One (next)	Two (next & prev)
Memory Usage	Less	More (extra pointer)
Traversal	One direction	Both directions
Insertion/Deletion	Slower (must track previous node)	Faster (direct prev access)

34. How do you insert a node at the end of a doubly linked list?

1. Create a new node.
2. Traverse to the last node.
3. Set `temp.next = new_node` and `new_node.prev = temp(last)`

Example:

```
def add_at_tail(self, new_data):  
    new_node = Node(new_data)  
  
    if not self.head: # If list is empty, new node becomes the head  
        self.head = new_node  
        return  
  
    temp = self.head  
    while temp.next: # Traverse to the last node  
        temp = temp.next  
  
    temp.next = new_node # Link last node to new node  
    new_node.prev = temp # Link new node to last node
```



35. What is the time complexity of deleting a node from a doubly linked list?

- **Searching for the node:** $O(n)$
- **Deleting the node:** $O(1)$ (adjust next and prev pointers)

✅ **Overall: $O(n)$ in worst case**

36. What is the advantage of a circular linked list over a singly linked list?

- **No Null Pointers:** The last node connects back to the first, reducing None checks.
- **Efficient Traversal:** Any node can be used as a starting point.
- **Good for Round-Robin Scheduling** (like CPU scheduling).

37. How do you create a circular linked list in Python?

1. Create a node.
2. Set `last.next = head` to make it circular.

Algorithm for Creating a Circular Linked List

1. **Create a new node** with given data.
2. **If the list is empty:**
 - Set `head = new_node`.
 - Point `new_node.next` to itself (circular link).
3. **If the list is not empty:**
 - Traverse to the last node.
 - Set `last.next = new_node`.
 - Set `new_node.next = head` to complete the circle.

✓ **Time Complexity:** $O(1)$ for empty list, $O(n)$ for insertion at the end.

38. What is the time complexity of traversing a circular linked list?

- Traversal is similar to a singly linked list.
- Stop when we reach the starting node again.

✓ **Time Complexity:** $O(n)$

39. What is garbage collection in Python?

Garbage collection is Python's **automatic memory management** that:

- Frees memory occupied by objects **no longer in use**.
- Uses **reference counting** and **cyclic garbage collection**.

✓ **Prevents memory leaks.**

40. What is the role of the gc module in Python?

The gc module provides:

- **Manual garbage collection control**
- **Checking memory leaks**

- **Disabling/enabling automatic garbage collection**
-

The End

